



MYSQL PROJECT REPORT

STUDENT NAME :	AAKASH NATH
AF CODE :	AF5170323
PROJECT TITLE :	THE CLOUD KITCHEN & FOOD DELIVERY MANAGEMENT SYSTEM
PROJECT DURATION :	3 DAYS
TRAINER NAME :	MR. AMITAVA CHATTERJEE
SUBMISSION DATE :	JULY , 2026
GITHUB USERNAME :	aakashnath
GITHUB REPOSITORY LINK :	<u>https://github.com/aakashnath/cloud-kitchen-food-delivery- management-system</u>

1. Project Overview

Online food delivery platforms have become a significant part of the modern food service industry, connecting customers, restaurants, and delivery personnel through a single digital ecosystem. Behind every such platform lies a relational database responsible for coordinating restaurant listings, menu items, customer orders, payment records, and delivery logistics in a consistent and reliable manner. **The Cloud Kitchen & Food Delivery Management System** project was undertaken to design and implement such a database from first principles, using **MySQL** as the underlying relational database engine.

Cloud kitchen platforms, unlike traditional restaurants, operate primarily through digital ordering channels rather than physical dine-in service. This business model places even greater importance on the underlying data infrastructure, since every stage of the customer journey, from browsing a menu to receiving a delivered order, depends entirely on accurate and synchronized data. A well-designed database is therefore central to ensuring that orders are processed correctly, payments are recorded reliably, and delivery status is tracked in real time.

This project addresses that requirement by building a normalized eleven-table relational schema that models the complete operational workflow of a cloud kitchen platform, from restaurant and menu management through to order placement, payment processing, delivery assignment, and post-delivery customer ratings. The database layer is further supported by triggers that encode business rules directly inside the database engine, and by reporting views that expose commonly required operational data without requiring repeated manual query construction.

To make the underlying database accessible and demonstrable, the project also includes a Streamlit-based interactive dashboard that visualizes key business metrics using Plotly charts, and allows direct interaction with the live database through dedicated pages for browsing tables, viewing reporting views, and executing custom SQL queries. Together, the database schema and the dashboard application present a complete, end-to-end demonstration of database management system concepts applied to a realistic business scenario.

2. Problem Statement

Coordinating restaurant listings, live customer orders, driver assignments, and restaurant rating calculations manually, or through loosely structured data storage, becomes increasingly error-prone as the volume of transactions grows. Without a properly normalized relational database, related pieces of information such as order status and delivery status can easily fall out of synchronization, leading to inconsistent or unreliable operational reporting. This project was undertaken to address a specific set of data-management challenges that arise in a cloud kitchen and food delivery business context.

A central challenge lies in maintaining consistent and accurate records for restaurants, menu items, customers, and delivery drivers, particularly as these entities are referenced repeatedly across orders, payments, and deliveries. Equally important is the need to track each order through its complete lifecycle, from the moment it is placed through preparation, dispatch, and final delivery or cancellation, while ensuring that the order status and the corresponding delivery status remain synchronized automatically rather than through separate, manually maintained updates.

Another significant challenge is the recalculation of restaurant ratings whenever new customer feedback is received. Performing this calculation manually, or repeatedly within application code, introduces both performance overhead and the risk of inconsistent results if the logic is implemented differently in different parts of the system. Similarly, producing live operational reports, such as a list of today's orders, currently pending deliveries, or the current top-performing restaurants, requires repeated construction of complex multi-table queries unless a dedicated reporting layer is established.

Finally, enforcing referential integrity across eleven interconnected tables demands careful schema design, since inconsistent foreign key relationships or missing constraints can allow invalid data to enter the system unnoticed. These combined challenges motivate the need for a properly normalized **MySQL** database with enforced constraints, automated trigger-based business logic, and a dedicated reporting layer, supported by an interactive dashboard capable of demonstrating these capabilities in an accessible manner.

3. Objectives and Scope

3.1 Objectives

The Cloud Kitchen & Food Delivery Management System was developed with the following specific objectives:

- Design a normalized eleven-table relational schema for a cloud kitchen platform.
- Enforce data integrity using primary keys, foreign keys, and CHECK constraints.
- Automate the order-delivered-to-rating-update business rule using MySQL triggers.
- Build a reporting layer using SQL Views for today's orders, top restaurants, and pending deliveries.
- Generate realistic sample data programmatically using the Faker library.
- Provide an interactive Streamlit and Plotly dashboard for business analytics.
- Provide a safe, ad-hoc SQL console restricted to custom SELECT queries.
- Migrate the system from SQLite to MySQL, resolving dialect differences in the process.

3.2 Scope

The scope of this project is limited to the design, implementation, and demonstration of the relational database and its accompanying dashboard application. It covers the complete data model required for a cloud kitchen platform, including customer management, restaurant and menu management, order processing, payment recording, delivery tracking, and post-delivery ratings. The project scope also includes the reporting views and trigger-based automation that operate directly within the database engine, along with the **Streamlit dashboard** used to visualize and interact with this data.

The project scope does not extend to production-level concerns such as user authentication, live payment gateway integration, real-time GPS-based driver tracking, or multi-user access control, as these fall outside the objectives of a database-focused academic project. The dataset used throughout the project is synthetic, generated using the Faker library, and is intended purely to demonstrate the structural and functional capabilities of the database design rather than to represent live production data.

4. System Requirements

4.1 Hardware Requirements

Component	Minimum Requirement
Operating System	Windows, macOS, or Linux
Memory (RAM)	Minimum 4 GB
Storage	Minimum 10 GB free disk space
Processor	Any modern dual-core processor or higher

4.2 Software Requirements

Software / Library	Version
MySQL Server	8.0.16 or higher (required for CHECK constraint enforcement)
Python	3.9 or higher
Streamlit	1.35 or higher
Pandas	2.0 or higher
Plotly	5.20 or higher
SQLAlchemy	2.0 or higher
PyMySQL	1.1 or higher
Faker	24.0 or higher

Software / Library	Version
python-dotenv	1.0 or higher

5. Tools and Technologies

This section describes each of the core technologies used in the Cloud Kitchen & Food Delivery Management System, along with the specific role each one plays within the project architecture.

5.1 MySQL

Definition: MySQL is an open-source relational database management system that organizes data into structured tables and enforces relationships between them using constraints.

Purpose: It provides the storage engine, constraint enforcement, and procedural capabilities such as triggers and views required to model complex, interrelated business data.

Role in This Project: MySQL 8.0 or higher serves as the primary database engine for this project, storing all eleven relational tables and enforcing CHECK constraints, AUTO_INCREMENT identity columns, business-logic triggers, and reporting views.

Advantages: MySQL offers mature support for referential integrity, a well-documented trigger and view syntax, and wide industry adoption, making it a realistic choice for demonstrating production-style database design.

5.2 Python

Definition: Python is a high-level, general-purpose programming language known for its readability and extensive ecosystem of libraries.

Purpose: It serves as the connective layer between the MySQL database and the user-facing dashboard, handling data retrieval, transformation, and presentation logic.

Role in This Project: Python is used to implement the Streamlit dashboard application, the database seeding script, and the shared database configuration module used across the project.

Advantages: Python's extensive library support, including SQLAlchemy, Pandas, and Faker, allows database connectivity, data processing, and synthetic data generation to be implemented efficiently within a single codebase.

5.3 Streamlit

Definition: Streamlit is a Python framework for building interactive, browser-based data applications with minimal front-end code.

Purpose: It provides the interactive dashboard interface through which users can view live database content, charts, and reporting views without writing SQL directly.

Role in This Project: Streamlit powers the eight-page dashboard application, including the sidebar navigation, KPI displays, data tables, and the trigger demonstration and custom SQL console pages.

Advantages: Streamlit allows a fully interactive, multi-page web application to be built directly in Python, avoiding the need for separate front-end web development while remaining suitable for demonstrating live database interaction.

5.4 Plotly

Definition: Plotly Express is a Python data visualization library that generates interactive charts such as bar charts, line charts, and donut charts.

Purpose: It converts tabular query results retrieved from the database into visual representations that make trends and comparisons easier to interpret.

Role in This Project: Plotly Express is used within the dashboard to render the orders-by-status donut chart, revenue-by-cuisine bar chart, thirty-day order trend line chart, payment-method distribution chart, and the top-restaurants horizontal bar chart.

Advantages: Plotly charts are interactive by default, allowing users to hover, zoom, and filter directly within the dashboard, which improves the accessibility of the underlying business data.

5.5 SQLAlchemy and PyMySQL

Definition: SQLAlchemy is a Python SQL toolkit that provides a database engine abstraction layer, while PyMySQL is a pure-Python MySQL database driver.

Purpose: Together they provide the connectivity mechanism between the Python application code and the MySQL database server.

Role in This Project: A cached SQLAlchemy engine is used within the dashboard application together with `pandas.read_sql_query` for executing queries, while PyMySQL is used directly as the raw DB-API driver within the data seeding script.

Advantages: This combination provides both a high-level, connection-pooled engine for the interactive application and a lightweight, direct driver for one-time bulk data seeding operations.

5.6 Faker

Definition: Faker is a Python library used to generate realistic synthetic data such as names, addresses, and transaction records.

Purpose: It populates the database with a representative volume of sample data so that the schema, triggers, and dashboard can be meaningfully demonstrated.

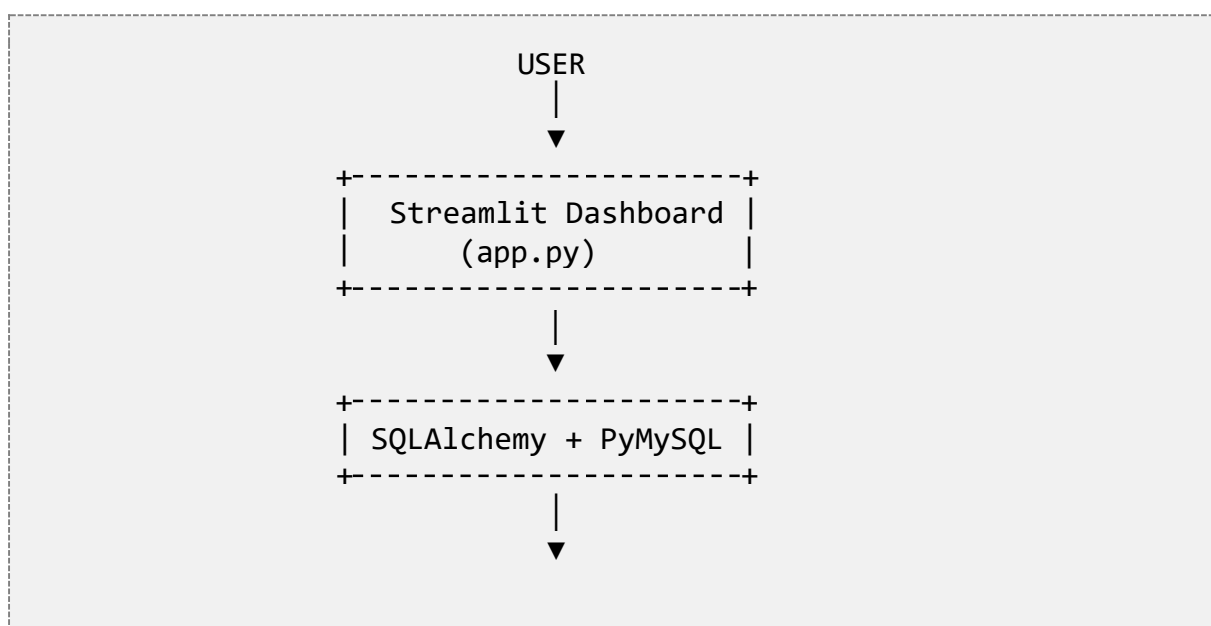
Role in This Project: Faker is used within the seeding script to generate sixty customers, fifteen restaurants, approximately one hundred and thirty-four menu items, twenty drivers, eight coupons, and two hundred and twenty complete orders spanning the full order-to-rating lifecycle.

Advantages: Faker allows realistic, varied sample data to be generated programmatically and reproducibly, avoiding the need to manually author large volumes of test data.

6. Project Architecture

The Cloud Kitchen & Food Delivery Management System follows a layered architecture in which each layer has a clearly defined responsibility. At the base of the architecture lies the MySQL database, which stores all relational data and enforces business rules through constraints, triggers, and views. Above this sits the application layer, implemented in Python, which connects to the database through SQLAlchemy and PyMySQL and exposes the data through the Streamlit dashboard. Users interact exclusively through the dashboard interface, which in turn communicates with the database layer to retrieve, display, and, where explicitly permitted, update data.

The overall data flow follows a consistent top-down pattern. A user interacting with the dashboard triggers a request that is routed through the cached SQLAlchemy engine to the MySQL database. The database processes the request, applying any relevant constraints or triggers, and returns the resulting data, which is then transformed using Pandas and rendered visually using Plotly charts or Streamlit data tables. This layered separation ensures that business logic enforced within the database, such as rating recalculation through triggers, remains independent of the presentation logic implemented in the dashboard.



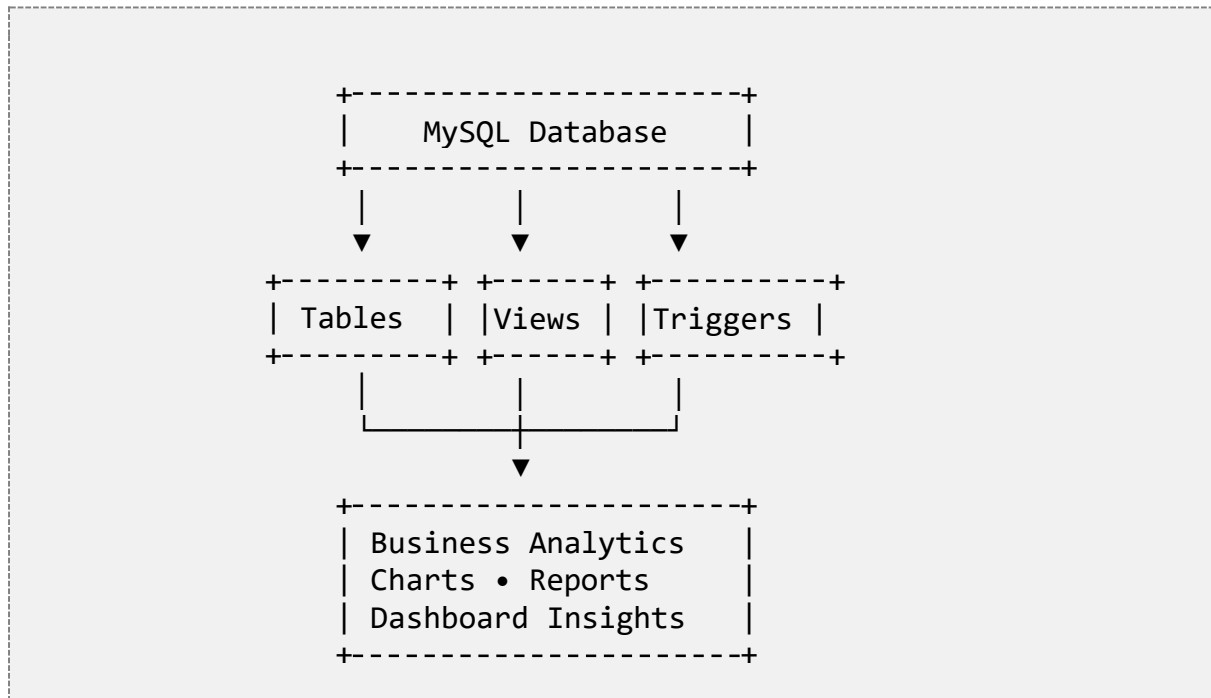


Figure 1: System Architecture Overview

7. Folder Structure

The project follows a clean, modular folder structure that separates the database layer from the application layer, making the codebase easy to navigate and maintain. The structure is presented below in a tree format.

```

cloud_kitchen_project/
├── database/
│   ├── schema.sql      -> Tables, triggers, and views (MySQL DDL)
│   ├── db_config.py    -> Shared MySQL connection configuration
│   └── seed_data.py     -> Truncates and repopulates sample data
├── app/
│   └── app.py           -> Streamlit dashboard application
├── requirements.txt     -> Python package dependencies
├── .env.example         -> Template for environment variables
└── README.md            -> Project documentation
  
```

The database folder contains all database-related artifacts. The `schema.sql` file defines the complete set of tables, triggers, and views using MySQL Data Definition Language, and must be executed first to establish the database structure. The `db_config.py` module centralizes the MySQL connection configuration, reading credentials from environment variables so that no sensitive information is hard-coded into the source files. The `seed_data.py` script uses the Faker library to generate and insert realistic sample records into the database, and can be re-run at any time to reset the data for demonstration purposes.

The app folder contains a single entry point, `app.py`, which implements the entire Streamlit dashboard, including page navigation, chart rendering, and the ad-hoc SQL console. The `requirements.txt` file lists all Python package dependencies required to run the project, while

the `.env.example` file provides a template that developers can copy and populate with their own MySQL credentials. The recommended setup sequence is to install the listed dependencies, configure the environment file, execute `schema.sql` to create the database structure, run `seed_data.py` to populate sample data, and finally launch the dashboard using the Streamlit command line interface.

8. Database Design Overview

The database underlying the **Cloud Kitchen & Food Delivery Management System** was planned around the natural entities present in a food delivery business, namely customers, restaurants, menu items, drivers, coupons, offers, orders, order line items, payments, deliveries, and ratings. Each entity was modeled as an independent table, with relationships between them expressed through foreign key constraints rather than through duplicated or embedded data, ensuring that every piece of information is stored in exactly one authoritative location.

The schema is designed to the third normal form, meaning that all non-key attributes depend only on the primary key of their respective table and not on other non-key attributes. As a practical example, order-item pricing is captured historically at the time an order is placed, rather than being derived from the current menu price, which preserves the accuracy of past orders even if menu prices change later. Similarly, payment and delivery records are enforced as strictly one-to-one with their parent order through UNIQUE foreign key constraints, reflecting the real-world rule that each order has exactly one payment transaction and exactly one delivery record.

Primary keys are defined on every table using auto-incrementing identity columns, ensuring that each record can be uniquely and efficiently referenced. Foreign keys are used extensively to connect related tables, for example linking Orders to Customers and Restaurants, and linking Order_Items to both Orders and Menus in a many-to-many junction pattern. Several foreign keys are defined with ON DELETE CASCADE behavior, such as the relationship between Restaurants and Menus, ensuring that dependent records are cleaned up automatically when a parent record is removed.

Database integrity is further reinforced through CHECK constraints that restrict specific columns to valid ranges or enumerated values. Examples include restricting restaurant and driver ratings to the range zero to five, restricting order status to a defined set of lifecycle values, and restricting payment method and payment status columns to a fixed list of acceptable values. All tables use the InnoDB storage engine, which is required for foreign key enforcement, together with utf8mb4 character encoding to ensure consistent handling of text data across the schema.

9. Database Tables

The database schema consists of eleven interconnected tables, each serving a distinct purpose within the overall data model. The table below summarizes the purpose, key fields, and constraints associated with each table.

Table	Purpose	Key Constraints
Customers	Stores end users who place orders	Primary key customer_id; UNIQUE email
Restaurants	Stores partner restaurants and cloud kitchens	CHECK rating between 0 and 5; is_active flag
Menus	Stores menu items offered by each restaurant	Foreign key to Restaurants (CASCADE); CHECK price > 0
Drivers	Stores delivery riders	CHECK rating between 0 and 5
Coupons	Stores platform-wide discount codes	UNIQUE code; CHECK discount between 0 and 100%
Offers	Stores restaurant-specific promotions	Foreign key to Restaurants (CASCADE)
Orders	Stores one record per customer order	Foreign keys to Customers, Restaurants, Coupons; CHECK status enum
Order_Items	Junction table linking Orders and Menus	Foreign keys to Orders (CASCADE) and Menus; CHECK quantity > 0
Payments	Stores one payment record per order	UNIQUE order_id; CHECK payment_method and payment_status enums
Deliveries	Stores one delivery record per order, linked to a driver	UNIQUE order_id; CHECK delivery_status enum
Ratings	Stores customer feedback submitted after delivery	UNIQUE order_id; CHECK rating_value between 1 and 5

9.1 Key Column Notes

- Orders.status is restricted using CHECK to the values Placed, Preparing, Out for Delivery, Delivered, and Cancelled.
- Deliveries.delivery_status is restricted using CHECK to the values Assigned, Picked Up, Delivered, Delayed, and Cancelled.
- Payments.payment_method is restricted using CHECK to the values UPI, Card, Cash, and Wallet.
- Payments.payment_status is restricted using CHECK to the values Pending, Success, Failed, and Refunded.

- Restaurants.rating and Drivers.rating are stored as DECIMAL(3,2) values bounded between 0 and 5 through CHECK constraints.
- Order_Items.item_price stores the price applicable at the time the order was placed, preserving historical accuracy independent of later menu price changes.

10. Entity Relationship Diagram

The Entity Relationship model provides a visual representation of how the eleven tables in the database relate to one another, making it easier to understand the overall data flow without examining the raw schema definitions directly. It is particularly useful for communicating the design to reviewers, evaluators, and other stakeholders who may not be directly familiar with the underlying SQL implementation.

10.1 Transaction Chain

```
Restaurants --< Menus
Restaurants --< Offers
Restaurants --< Orders
Customers --< Orders
Customers --< Ratings

Orders --< Order_Items >-- Menus
Orders --1:1-- Payments
Orders --1:1-- Deliveries --< (many) Drivers
Orders --1:1-- Ratings
Coupons --< Orders (nullable foreign key)
```

10.2 Key Relationships

Table	Relationship
Restaurants	One-to-many with Menus, Offers, and Orders
Customers	One-to-many with Orders and Ratings
Orders	Many-to-one with Customer, Restaurant, and Coupon (nullable)
Order_Items	Many-to-one with Order and many-to-one with Menu (junction table)
Payments	One-to-one with Order
Deliveries	One-to-one with Order and many-to-one with Driver
Ratings	One-to-one with Order, many-to-one with Customer and Restaurant

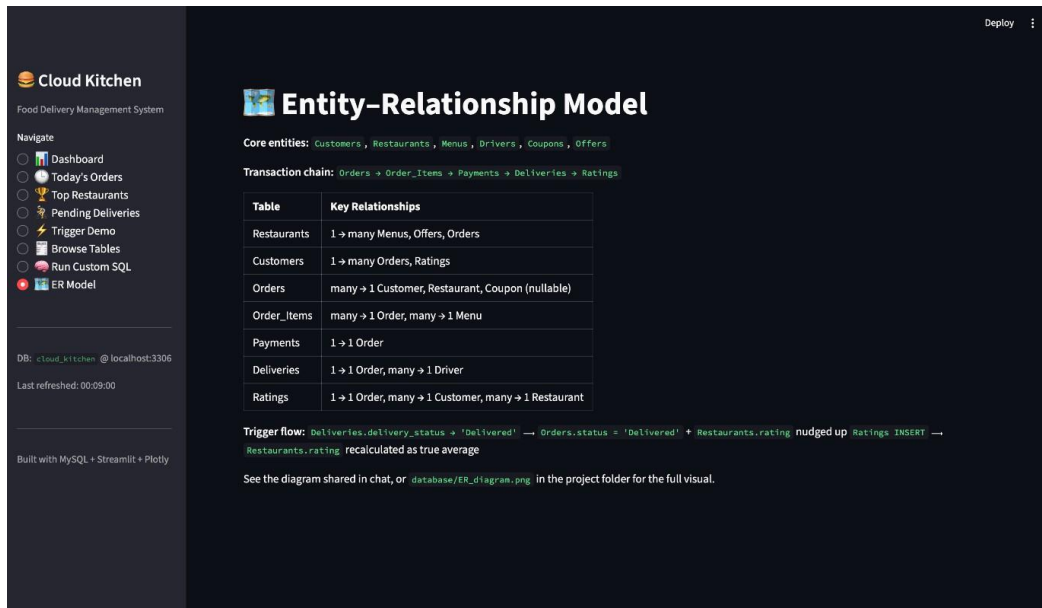


Figure 2: Entity Relationship Model

This design is fully normalized to the third normal form: order-item pricing is captured historically, payments and deliveries are enforced as strictly one-to-one with orders through UNIQUE foreign keys, and lookup data such as cuisine type, city, and vehicle type is stored directly rather than duplicated across tables.

11. SQL Concepts Demonstrated

This project demonstrates a broad range of core SQL concepts, each applied in a specific, practical context within the database schema. The following subsections describe each concept, its purpose, and how it is used within this project.

11.1 Data Definition Language (DDL)

Definition: Data Definition Language refers to SQL statements used to define and modify the structure of database objects such as tables, triggers, and views.

Purpose: DDL statements establish the schema within which all data will be stored and enforce structural rules such as constraints and relationships.

Project Usage: schema.sql uses CREATE DATABASE and CREATE TABLE statements to define all eleven tables, along with CREATE TRIGGER and CREATE VIEW statements for the automation and reporting layers. DROP TABLE, DROP TRIGGER, and DROP VIEW statements are used where schema objects need to be reset.

Benefits: Defining the schema explicitly through DDL ensures that structural rules, such as data types and constraints, are enforced consistently by the database engine rather than relied upon at the application level.

11.2 Data Manipulation Language (DML)

Definition: Data Manipulation Language refers to SQL statements used to insert, update, or remove data within existing tables.

Purpose: DML statements are used to populate the database with records and to modify existing records as business events occur.

Project Usage: INSERT statements are used extensively within seed_data.py to populate all eleven tables with synthetic data. UPDATE statements are used both directly and within trigger bodies, for example when a delivery status changes or when a restaurant's rating is recalculated. TRUNCATE TABLE statements are used to reset all tables before reseeding.

Benefits: DML provides the mechanism through which the database is kept current and reflects real business events, such as new orders or updated delivery statuses.

11.3 Data Query Language (DQL)

Definition: Data Query Language refers to the SELECT statement and its associated clauses, used to retrieve data from one or more tables.

Purpose: DQL statements allow specific subsets of data to be retrieved, filtered, grouped, and ordered according to business requirements.

Project Usage: SELECT statements combined with WHERE, GROUP BY, and ORDER BY clauses are used throughout the reporting views and the dashboard application. Aggregate functions such as COUNT(), SUM(), and AVG() are used to compute metrics including total orders, total revenue, and average ratings.

Benefits: DQL enables flexible, on-demand retrieval of exactly the data required for a given report or dashboard visualization, without duplicating that data elsewhere.

11.4 Joins

Definition: A join combines rows from two or more tables based on a related column between them.

Purpose: Joins allow related data stored across separate normalized tables to be retrieved together in a single result set.

Project Usage: INNER JOIN is used to combine customer and restaurant details with order records, while LEFT JOIN is used in vw_top_restaurants to include restaurants even when they have no associated orders, and in vw_pending_deliveries to include deliveries even when no driver has yet been assigned.

Benefits: Joins allow the normalized schema to be queried as if it were a single unified view of the business, without sacrificing the data-integrity benefits of normalization.

11.5 Views

Definition: A view is a stored, named SELECT query that can be queried like a regular table without re-executing the underlying joins manually each time.

Purpose: Views provide a reusable reporting layer that decouples reporting logic from the application code that consumes it.

Project Usage: Three views are implemented in this project: `vw_todays_orders`, `vw_top_restaurants`, and `vw_pending_deliveries`, each described in detail in the following section.

Benefits: Views simplify the dashboard application code, since each dashboard page can query a single view rather than reconstructing complex multi-table joins directly within Python.

11.6 Triggers

Definition: A trigger is a stored procedure that executes automatically in response to a specified data modification event on a table.

Purpose: Triggers allow business rules to be enforced consistently at the database level, regardless of which application or client modifies the data.

Project Usage: Two triggers are implemented in this project, `trg_order_delivered_bump_rating` and `trg_rating_recalc_restaurant_avg`, both described in detail in the following section on trigger-based automation.

Benefits: Encoding business logic within triggers ensures that the rating-update rule cannot be bypassed or implemented inconsistently across different parts of the application.

11.7 Stored Procedures

Definition: A stored procedure is a named, reusable block of procedural SQL logic that can be invoked explicitly by an application or user.

Purpose: Stored procedures allow multi-step procedural logic to be encapsulated and executed on demand.

Project Usage: This project does not implement stored procedures. Its business logic is instead encoded through triggers, which execute automatically in response to data events, and through reporting views, which encapsulate reusable queries.

Benefits: While not used in this project, stored procedures remain a natural extension for future development, particularly for encapsulating multi-step administrative operations that fall outside the current SELECT-only console.

11.8 Constraints and Functions

The schema also makes use of PRIMARY KEY and AUTO_INCREMENT constraints for identity management, FOREIGN KEY constraints with ON DELETE CASCADE for referential integrity, UNIQUE constraints to enforce one-to-one relationships and unique codes, and CHECK constraints to restrict columns to valid ranges or enumerated values. Built-in SQL functions used throughout the project include LEAST() for capping the automatic rating bump, ROUND() for rating precision, CURDATE() and CURRENT_TIMESTAMP for date and time handling, and COALESCE() for handling restaurants with no recorded orders.

12. Trigger-Based Business Logic

The rating-automation feature is the centerpiece of this project's database design. Two chained MySQL triggers implement the complete order-delivered-to-rating-update business rule entirely inside the database engine, without requiring any rating computation to be performed at the application level.

12.1 Trigger 1: trg_order_delivered_bump_rating

This trigger fires after an UPDATE operation on the Deliveries table. Since MySQL does not support an AFTER UPDATE OF <column> clause, unlike SQLite or PostgreSQL, the column-change check is performed explicitly using an IF statement within the trigger body.

```
CREATE TRIGGER trg_order_delivered_bump_rating
AFTER UPDATE ON Deliveries
FOR EACH ROW
BEGIN
  IF NEW.delivery_status = 'Delivered'
    AND OLD.delivery_status != 'Delivered' THEN

    UPDATE Orders SET status = 'Delivered'
    WHERE order_id = NEW.order_id;

    UPDATE Restaurants
    SET rating = LEAST(5.0, ROUND(rating + 0.01, 2))
    WHERE restaurant_id = (
      SELECT restaurant_id FROM Orders
      WHERE order_id = NEW.order_id
    );
  END IF;
END
```

Effect: As soon as a delivery record changes to the Delivered status, the parent order is automatically marked Delivered and the associated restaurant receives a small automatic rating increase, capped at 5.0 using the LEAST() function, serving as an immediate success signal.

12.2 Trigger 2: trg_rating_recalc_restaurant_avg

This trigger fires after an INSERT operation on the Ratings table, that is, at the moment a customer actually submits a star rating for a completed order.

```
CREATE TRIGGER trg_rating_recalc_restaurant_avg
AFTER INSERT ON Ratings
FOR EACH ROW
BEGIN
    UPDATE Restaurants
    SET rating = ROUND((
        SELECT AVG(rating_value) FROM Ratings
        WHERE restaurant_id = NEW.restaurant_id
    ), 2)
    WHERE restaurant_id = NEW.restaurant_id;
END
```

Effect: This trigger recalculates the restaurant's rating as the true average of every rating it has ever received, overriding the earlier temporary bump applied by the first trigger with an accurate, data-backed figure.

Together, this two-stage design, an instant provisional nudge followed by a true recalculation once real feedback is submitted, demonstrates deliberate business-logic modeling rather than a single trivial trigger. This behavior is best demonstrated live through the dashboard's dedicated Trigger Demo page, described later in this report.

13. SQL Views (Reporting Layer)

Three reporting views were implemented to expose commonly required operational data directly, without requiring the dashboard application to reconstruct complex multi-table joins on every request.

13.1 vw_todays_orders

Returns all orders placed on the current date, joined with the corresponding customer and restaurant names.

```
CREATE VIEW vw_todays_orders AS
SELECT o.order_id, c.name AS customer_name,
       r.name AS restaurant_name, o.status,
       o.total_amount, o.order_date
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
WHERE DATE(o.order_date) = CURDATE()
ORDER BY o.order_date DESC;
```

13.2 vw_top_restaurants

Ranks restaurants by rating, with order volume used as a tie-breaker, alongside their total order count and total revenue.


```
CREATE VIEW vw_top_restaurants AS
SELECT r.restaurant_id, r.name, r.cuisine, r.city, r.rating,
       COUNT(o.order_id) AS total_orders,
       COALESCE(SUM(o.total_amount),0) AS total_revenue
FROM Restaurants r
LEFT JOIN Orders o ON o.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name, r.cuisine, r.city, r.rating
ORDER BY r.rating DESC, total_orders DESC;
```

13.3 vw_pending_deliveries

Returns all deliveries that have not yet reached the Delivered or Cancelled status, joined across Orders, Customers, Restaurants, and Drivers.

```
CREATE VIEW vw_pending_deliveries AS
SELECT d.delivery_id, o.order_id, c.name AS customer_name,
       r.name AS restaurant_name, dr.name AS driver_name,
       d.delivery_status, d.pickup_time, o.order_date
FROM Deliveries d
JOIN Orders o ON o.order_id = d.order_id
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Restaurants r ON r.restaurant_id = o.restaurant_id
LEFT JOIN Drivers dr ON dr.driver_id = d.driver_id
WHERE d.delivery_status NOT IN ('Delivered','Cancelled')
ORDER BY o.order_date ASC;
```

14. Sample Data Generation

The seed_data.py script truncates all eleven tables and repopulates them each time it is executed, which is particularly useful for resetting the Trigger Demo page once every delivery in the dataset has been marked Delivered. The Faker library is used throughout this script to generate realistic, varied sample records rather than repetitive placeholder values.

Entity	Volume
Customers	60
Restaurants	15
Menu items	6 to 12 per restaurant, approximately 134 in total
Drivers	20
Coupons	8
Offers	0 to 2 per restaurant
Orders	220, each with a complete order-to-payment-to-delivery-to-rating lifecycle

15. Dashboard Modules

The Streamlit dashboard application (app/app.py) uses a cached SQLAlchemy engine together with a shared pair of run_query() and run_write() helper functions, and presents its

functionality across eight distinct pages accessible through a sidebar radio-button navigator. Before rendering any page, the application performs a startup safety check by attempting a test connection through the cached engine; if the connection fails, it displays a descriptive error message, for example indicating a missing .env file, an unapplied schema, or an unseeded database, and halts gracefully rather than crashing. Each dashboard module is described in detail below, together with a placeholder for the corresponding screenshot.

15.1 Business Dashboard Overview

The main Dashboard page presents a high-level operational snapshot through a set of key performance indicator cards, including Total Orders, Delivered orders, Cancelled orders, total Revenue, and Average Rating across the platform. These KPI cards allow a business stakeholder to assess the overall health of the platform at a glance, without needing to interpret any underlying chart or table.



Figure 3: Business Dashboard Overview

Business Importance: The KPI card layout is designed for executive-level consumption, allowing leadership to assess order volume, cancellation rate, revenue, and customer satisfaction trends within a single view, supporting rapid, high-level decision-making without requiring direct database access.

15.2 Order Trends and Payment Analytics

Alongside the KPI cards, the Dashboard page presents a set of Plotly visualizations, including an orders-by-status donut chart, a revenue-by-cuisine bar chart, a thirty-day order trend line chart, and a payment-method distribution chart. These visualizations reveal patterns that the KPI cards alone cannot, such as which cuisines generate the most revenue and how order volume fluctuates on a day-to-day basis.



Figure 4: Order Trends and Payment Analytics

Business Importance: These visualizations support content and marketing decisions, for example identifying which cuisines to promote further, and support financial reconciliation by revealing the relative share of each payment method used across the platform.

15.3 Today's Orders SQL View

The Today's Orders page presents a live, filterable view of the `vw_todays_orders` reporting view, allowing users to see every order placed on the current date, along with the associated customer name, restaurant name, order status, and total amount. Users can filter the displayed results by order status.

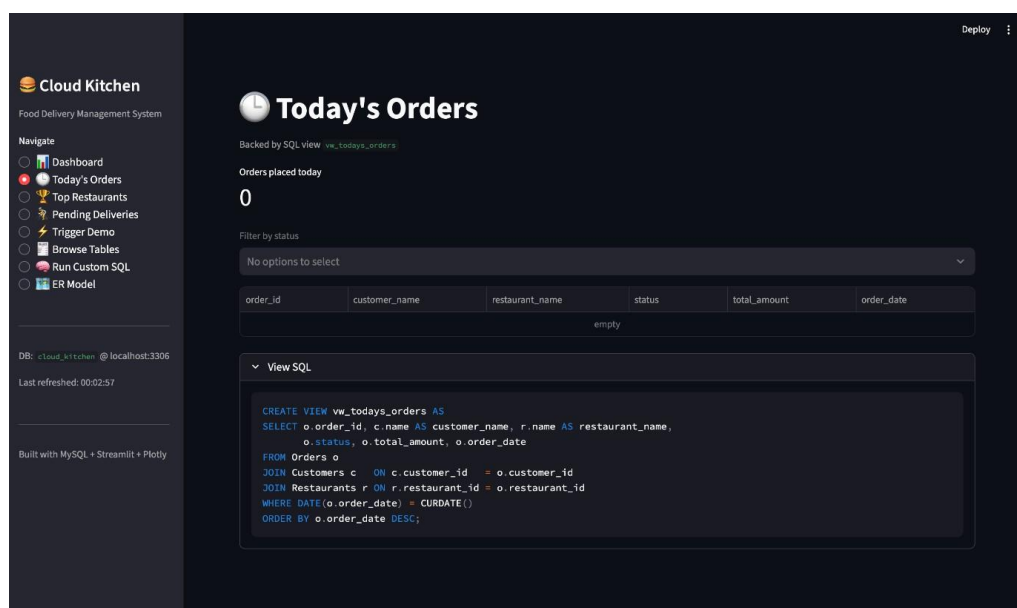


Figure 5: Today's Orders SQL View

Business Importance: This page provides operations staff with immediate visibility into the current day's order activity, supporting real-time monitoring of order volume and status without requiring direct SQL access to the underlying database.

15.4 Top Restaurants Dashboard

The Top Restaurants page presents a live view of the `vw_top_restaurants` reporting view, ranking restaurants by rating with order volume as a tie-breaker. An adjustable Top-N slider allows the user to control how many restaurants are displayed, and the results are also rendered as a horizontal bar chart for easier visual comparison.

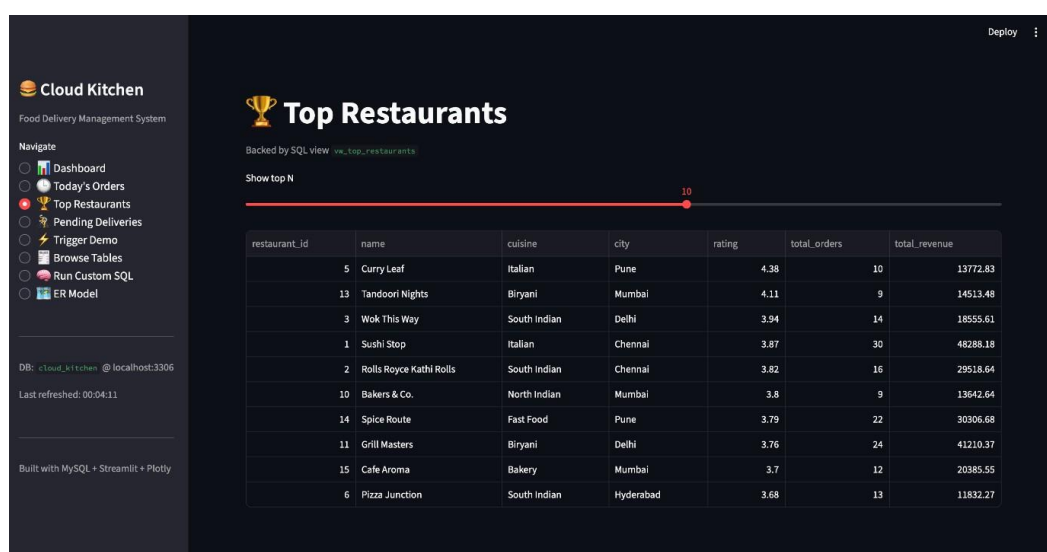


Figure 6: Top Restaurants Dashboard

Business Importance: This page directly supports regional and partnership planning by identifying which restaurants are performing best in terms of both customer satisfaction and order volume, informing decisions about which partners to prioritize for promotional support.

15.5 Restaurant Rating Visualization

Restaurant ratings are surfaced visually across multiple points in the dashboard, most notably within the Top Restaurants horizontal bar chart, where restaurants are ordered by their current rating value. Because ratings are recalculated automatically by the underlying database triggers, the values displayed here always reflect the true, data-backed average rather than a manually maintained figure.

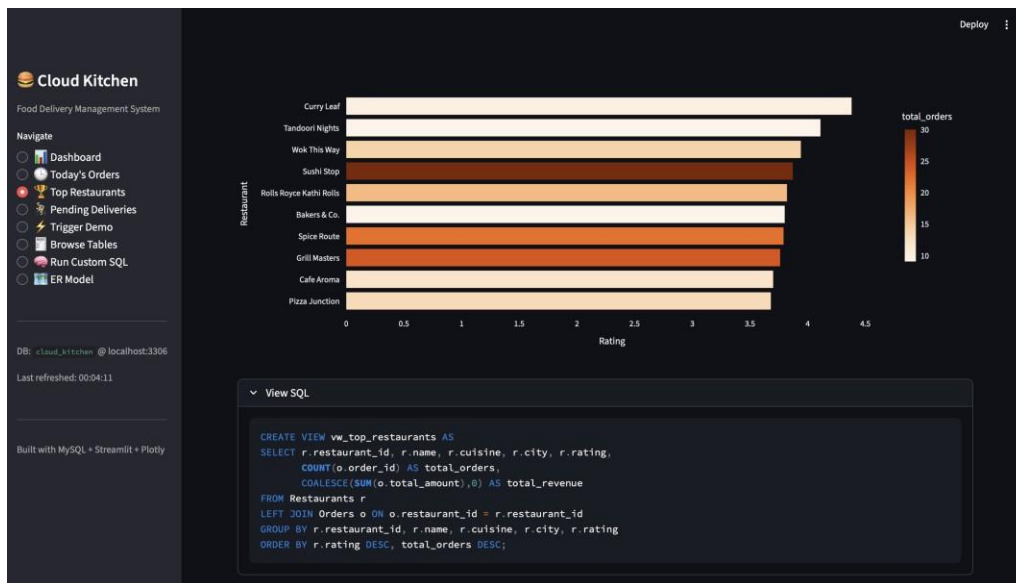


Figure 7: Restaurant Rating Visualization

Business Importance: Visualizing ratings alongside order volume helps distinguish restaurants that are merely popular from those that are both popular and consistently well rated, which is a more reliable signal for long-term partnership decisions.

15.6 Pending Deliveries Monitoring

The Pending Deliveries page presents a live view of the `vw_pending_deliveries` reporting view, listing every delivery that has not yet reached the Delivered or Cancelled status, joined with the associated customer, restaurant, and driver names. An in-progress delivery counter is displayed alongside the table to summarize the current operational load.

The screenshot displays the Cloud Kitchen dashboard with the 'Pending Deliveries' page active. The page title is 'Pending Deliveries' and it is backed by the SQL view `vw_pending_deliveries`. It shows 'Deliveries in progress' with a counter of 23. Below this is a table with the following columns: `delivery_id`, `order_id`, `customer_name`, `restaurant_name`, `driver_name`, `delivery_status`, `pickup_time`, and `order_date`.

delivery_id	order_id	customer_name	restaurant_name	driver_name	delivery_status	pickup_time	order_date
66	70	Gavin Walli	The Biryani House	Meghana Mukherjee	Picked Up	2026-07-19 05:49:47	2026-07-19 05:36:47
144	151	Onveer Walla	Sushi Stop	Gauri Bumb	Assigned	None	2026-07-19 06:36:47
63	67	Frederick Khurana	Wok This Way	Hardik Nayak	Assigned	None	2026-07-19 09:36:47
9	9	Inaya Kapur	Green Bowl	Januja Pradhan	Assigned	None	2026-07-19 11:00:47
10	10	Onveer Walla	Bakers & Co.	Madhav Kunda	Assigned	None	2026-07-19 11:13:47
102	109	Lajita Seshadri	Pizza Junction	Januja Pradhan	Picked Up	2026-07-19 12:55:47	2026-07-19 12:43:47
53	56	Lipika Setty	Noodle Bar	Victor Tella	Picked Up	2026-07-19 13:29:47	2026-07-19 13:08:47
32	33	Farhan Memon	Sushi Stop	Gayathri Deep	Assigned	None	2026-07-19 13:26:47
62	66	Charan Jani	Grill Masters	Hardik Nayak	Picked Up	2026-07-19 13:47:47	2026-07-19 13:33:47
31	32	Darpan Gandhi	Kebab Factory	Lakshit Krishna	Assigned	None	2026-07-19 13:42:47

At the bottom of the table, there is a 'View SQL' button.

Figure 8: Pending Deliveries Monitoring

Business Importance: This page gives operations staff a real-time view of deliveries currently in progress, supporting proactive identification of delayed deliveries before they affect customer satisfaction.

15.7 Trigger Demonstration

The Trigger Demo page allows a user to select a specific delivery record and mark it as Delivered directly through the dashboard interface. Upon this action, the two chained MySQL triggers described earlier in this report execute automatically, first marking the parent order as Delivered and applying a small rating bump, and then, once a rating is submitted, recalculating the restaurant's true average rating. The updated rating is reflected live on this page immediately after the action is performed.

Cloud Kitchen
Food Delivery Management System

Navigate

- Dashboard
- Today's Orders
- Top Restaurants
- Pending Deliveries
- Trigger Demo**
- Browse Tables
- Run Custom SQL
- ER Model

DB: cloud_kitchen @ localhost:3306
Last refreshed: 00:06:09

Built with MySQL + Streamlit + Plotly

⚡ Trigger Demo: Order Delivered → Increase Restaurant Rating

This page lets you fire the trigger live. Pick a delivery that isn't yet **Delivered**, mark it delivered, and watch the restaurant's rating update automatically — no application code computes the new rating, the database trigger does it.

> View trigger SQL

delivery_id	order_id	restaurant_name	rating	customer_name	delivery_status
149	156	Pizza Junction	3.68	Meghana Shanker	Picked Up
169	180	Sushi Stop	3.87	Yuvraj Loke	Assigned
70	74	Sushi Stop	3.87	Sachi Borah	Assigned
12	12	Wok This Way	3.94	Aarnav Mandal	Picked Up
148	155	Sushi Stop	3.87	Varsha Ahuja	Assigned
132	139	Curry Leaf	4.38	Bhavya Rattan	Assigned
196	207	Noodle Bar	3.52	Harini Gour	Assigned
183	194	Sushi Stop	3.87	Rehaan Salvi	Assigned
163	174	Sushi Stop	3.87	Darsh Rao	Assigned
41	42	The Biryani House	3.58	Darpan Gandhi	Picked Up

Pick a delivery to mark as Delivered

Delivery #149 — Order #156 — Pizza Junction (current rating 3.68)

Mark as Delivered (fire trigger)

Figure 9: Trigger Demonstration Interface

Business Importance: This page is the clearest practical demonstration of the project's core value proposition, showing evaluators and stakeholders that business rules are enforced automatically and reliably at the database level rather than depending on correct application-side logic.

15.8 Browse Raw Database Tables

The Browse Tables page provides a raw, tabular viewer for all eleven tables in the database schema, with an adjustable row limit control that allows users to control how many records are displayed at a time. This page is intended primarily for verification and inspection purposes rather than day-to-day operational use.

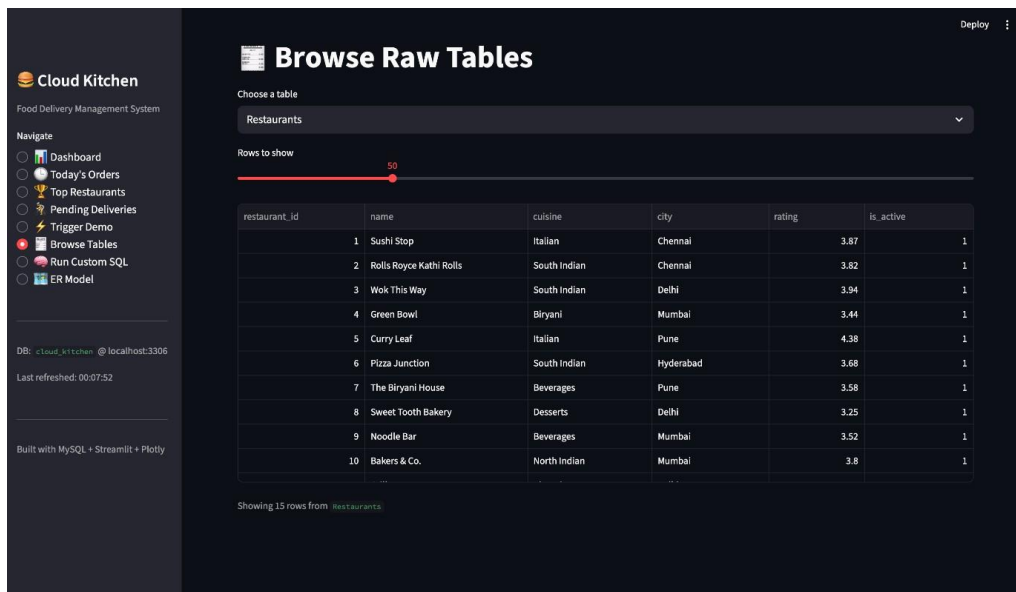


Figure 10: Browse Raw Database Tables

Business Importance: This page supports transparency and verification, allowing evaluators and reviewers to directly inspect the raw underlying data behind every chart and KPI presented elsewhere in the dashboard.

15.9 Custom SQL Query Execution

The Run Custom SQL page provides an ad-hoc query console that allows users to type and execute their own SELECT statements directly against the live database. The console is deliberately restricted to SELECT-only queries by design, preventing accidental or unauthorized modification of the underlying data while still allowing flexible, exploratory analysis.

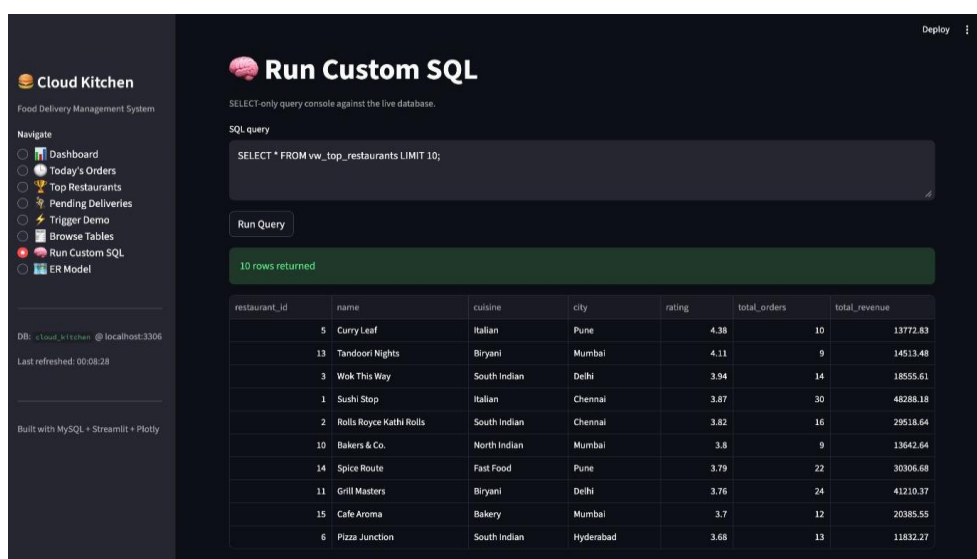


Figure 11: Custom SQL Query Execution

Business Importance: This page allows technically capable stakeholders to answer bespoke analytical questions that are not covered by the predefined dashboard pages, without granting write access to the production data.

The dashboard application additionally includes an ER Model page, providing an in-app relationship summary and trigger-flow explanation that mirrors the Entity Relationship Model presented earlier in this report, allowing users to review the schema design without leaving the application.

16. Dashboard Features Summary

The table below summarizes the eight pages of the Streamlit dashboard application and the primary feature offered by each.

Page	Primary Feature
Dashboard	KPI cards, orders-by-status donut chart, revenue-by-cuisine chart, 30-day trend, payment-method split
Today's Orders	Live filterable view of vw_todays_orders by order status
Top Restaurants	Live view of vw_top_restaurants with adjustable Top-N slider and bar chart
Pending Deliveries	Live view of vw_pending_deliveries with in-progress delivery counter
Trigger Demo	Mark a delivery Delivered and observe automatic rating update via triggers
Browse Tables	Raw viewer for all 11 tables with adjustable row limit
Run Custom SQL	SELECT-only ad-hoc query console against the live database
ER Model	In-app relationship summary and trigger-flow explanation

17. SQLite to MySQL Migration Notes

The project was originally implemented in SQLite and subsequently migrated to MySQL, which added a genuine software-engineering dimension to the work beyond a typical single-database classroom exercise. The migration required changes across the schema definition, the data seeding script, and the application connection layer.

17.1 Changes to schema.sql

- AUTOINCREMENT was replaced with AUTO_INCREMENT.
- REAL data types were replaced with DECIMAL.
- Explicit CREATE DATABASE and USE statements were added.

- Triggers were rewritten using DELIMITER \$\$... END\$\$ blocks, with an explicit IF statement added for column-change detection, since MySQL has no AFTER UPDATE OF <column> clause.
- date('now') was replaced with CURDATE(), and MIN(5.0, ...) was replaced with LEAST(5.0, ...).

17.2 Changes to seed_data.py

- The sqlite3 driver was replaced with pymysql.
- Question-mark ('?') placeholders were replaced with '%s' placeholders.
- File-based database reset was replaced with TRUNCATE TABLE statements, with the schema and triggers assumed to already exist via schema.sql.

17.3 Changes to app.py

- The raw sqlite3 connection was replaced with a cached SQLAlchemy engine defined in db_config.py.
- The same run_query() and run_write() helper function shape was retained to minimize code churn.
- Question-mark ('?') placeholders were replaced with '%s' placeholders.
- date('now', '-30 day') was replaced with CURDATE() - INTERVAL 30 DAY.
- The startup safety check now pings the live MySQL database instead of checking for the presence of a local .db file.
-

18. Key Insights

Several practical insights emerged from the design and implementation of this project, illustrating how database-level automation can directly support day-to-day business operations.

- Encoding the rating-update rule directly within database triggers removes the risk of inconsistent rating calculations across different parts of the application.
- A two-stage trigger design, consisting of an immediate provisional update followed by a true recalculation, more accurately reflects the way rating systems behave in real platforms.
- Reporting views such as vw_todays_orders and vw_pending_deliveries significantly simplify the dashboard's application code by removing the need to reconstruct complex joins repeatedly.
- Enforcing one-to-one relationships between Orders, Payments, and Deliveries through UNIQUE foreign keys prevents data anomalies such as duplicate payment records for a single order.
- Migrating the project from SQLite to MySQL revealed practical, non-obvious differences in trigger syntax and date-handling functions that are easy to overlook when designing for a single database engine.

19. Business Value

The Cloud Kitchen & Food Delivery Management System delivers meaningful business value across several dimensions of operational decision-making. At the database level, enforced referential integrity and CHECK constraints prevent invalid or inconsistent data from entering the system, reducing the risk of operational errors that would otherwise require manual correction. The trigger-based rating automation ensures that restaurant ratings remain accurate and current at all times, without requiring dedicated application logic or scheduled batch jobs to maintain them.

At the dashboard level, the KPI cards and visual summaries provide a quick, high-level overview that enables leadership to assess the platform at a glance, while the dedicated reporting pages, including Today's Orders, Top Restaurants, and Pending Deliveries, support day-to-day operational monitoring without requiring direct SQL access. The Run Custom SQL console extends this further by allowing technically capable stakeholders to answer bespoke analytical questions safely, since it is restricted to SELECT-only queries.

Collectively, the combination of a rigorously normalized database, automated business-rule enforcement, and an accessible dashboard interface allows the system to function as a reliable tool for both operational management and business intelligence reporting, closely mirroring the kind of infrastructure required by real cloud kitchen and food delivery platforms.

20. Advantages

- Fully normalized eleven-table schema with enforced referential integrity.
- Business logic for rating updates is automated entirely at the database level via triggers.
- Reusable reporting layer implemented through SQL Views, decoupled from application code.
- Interactive, real-time dashboard accessible to non-technical stakeholders.
- Safe, read-only SQL console available for ad-hoc analysis.
- Realistic, reproducible sample data generation using the Faker library.
- Demonstrates a genuine cross-database migration from SQLite to MySQL.

21. Limitations

- Uses synthetic, demonstration data rather than live production data.
- No authentication or role-based access control; the application presents a single, admin-style interface.
- No integration with a real payment gateway.
- No live GPS-based driver-tracking capability.

- The custom SQL console is deliberately restricted to SELECT-only queries, which limits deeper ad-hoc administrative actions.

22. Future Scope

- Customer-facing and restaurant-facing login portals with role-based access control.
- A driver mobile application with live GPS tracking capability.
- Integration with a real payment gateway for live transaction processing.
- SMS and email notifications for order and delivery status changes.
- An AI-based menu and restaurant recommendation engine.
- Machine-learning based demand forecasting.
- Cloud deployment using a managed MySQL instance together with a containerized Streamlit application.

23. Learning Outcomes

Through the design, development, and documentation of this project, the following practical skills and areas of understanding were developed.

23.1 Technical Skills

- Designing and implementing a normalized relational database schema in MySQL.
- Writing and debugging MySQL triggers, including handling dialect-specific syntax differences.
- Building reusable SQL Views for reporting purposes.
- Developing an interactive multi-page dashboard using Streamlit and Plotly.
- Generating realistic synthetic datasets using the Faker library.

23.2 Database Concepts

- Normalization principles and their practical application to a real schema.
- Enforcing referential integrity through primary keys, foreign keys, and CHECK constraints.
- Understanding the distinction between DDL, DML, and DQL operations.
- Applying joins, aggregate functions, and grouping to construct meaningful reports.

23.3 Software Development and Documentation Experience

- Managing a cross-database migration from SQLite to MySQL, including connection-layer and syntax changes.
- Structuring a project codebase into clearly separated database and application layers.
- Preparing professional technical documentation suitable for academic and industry review.

24. Conclusion

The Cloud Kitchen & Food Delivery Management System demonstrates a complete and realistic application of relational database design and automation principles to a genuine business scenario. Its eleven-table normalized schema, CHECK-constrained data integrity, two-stage trigger chain, and reporting views collectively encode real business rules directly inside **MySQL**, while the accompanying **Streamlit** dashboard exposes that data through live **KPIs**, interactive charts, and a controlled ad-hoc SQL console.

The migration of the project from SQLite to MySQL adds a practical software-engineering dimension, illustrating how trigger syntax, data types, and connection handling differ meaningfully across database engines, and how such differences must be resolved deliberately during a real migration rather than assumed to be trivial.

Overall, this project serves as a strong demonstration of how core database management system concepts, including normalization, joins, aggregate functions, views, triggers, and constraints, can be combined with a modern data application to deliver both reliable operational functionality and meaningful business intelligence.

TRAINER VERIFICATION SECTION

Trainer Remarks: _____

Signature: _____

Date: _____ / _____ / _____